

JAVA程序设计



第6讲 内容向导

6.1 抽象类

6.2 最终类

6.3 接口



6.1 抽象类

➤ abstract——抽象类

- 不能实例化，即不能创建对象, 主要作为基类或父类存在
- 必须实现所有abstract方法



➤ 抽象方法的性质

- (1) 由于抽象方法在抽象类中没有具体的方法实现，所以抽象方法必须被子类覆盖。抽象方法不能为**private**。
- (2) 不能将构造方法、类成员方法声明为抽象方法。
- (3) **abstract**抽象方法不能和**static**同用。
- (4) 父类的非抽象方法可以被子类的抽象方法覆盖。



➤ 实例

- 父类: **Shape** 类

- `import java.util.*;`

- `abstract class Shape {`

- `public abstract void Area();//没有实现`

-

- }



- 子类:**Rectangle**类是**Shape**类的一个子类

- **class Rectangle extends Shape {**
- **int a,b;**
- **Rectangle(int aa,int bb) {a=aa;b=bb; }**
- **public void Area() { //子类来实现**
- **System.out.println("矩形面积:"+a*b);**
- **}**



- 子类:**Circle**类也是**Shape**类的子类

- **class Circle extends Shape{**

- **int r;**

- **Circle(int rr){r=rr;}**

- **public void Area() { //子类来实现**

- **System.out.println("圆:"+Math.PI*r*r);**

- **}**

- **}**



- 执行类:
- public class less {
- public static void main(String[] args) {
-
- Shape[] shapes = new Shape[4];
- shapes[0]=new Rectangle(2,4);
- shapes[1]=new Rectangle(3,5);
- shapes[2]=new Circle(1);
- shapes[3]=new Circle(2);
- for(int i=0;i<shapes.length;i++){
- shapes[i].Area(); //体现多态
- }
- }



6.2 最终类

➤ final——最终类，不能被继承

```
public final class Test {
```

```
.....
```

```
}
```

注意：声明类时abstract和final不能同时使用



6.3 接口

➤ 接口

- 接口 **interface**——方法定义和常量值的集合
- 接口是特殊的抽象类
- 只包含常量和方法的定义，而没有方法的实现



➤ 接口声明

interface 接口名字

```
{ ..... //常量定义和方法定义  
}
```

```
interface Computable
```

```
{ final int MAX=100;
```

```
    void add();
```

```
    float sum(float x, float y);
```

```
}
```



➤接口使用

关键字——implements

例如: **class A implements Printable , Addable**

说明: 类A中使用接口**Printable** 和 **Addable**



注意事项:

- ✓ 类体中必须实现接口中定义的所有方法;
- ✓ 接口中的方法和常量必须是**public**的。
- ✓ 实现方法时，方法头必须一致（返回值类型，名字，参数）；
- ✓ 父类被继承时，接口同时被继承；
- ✓ 接口也可被继承，关键字为**extends**；
- ✓ 接口一般表示功能，而类一般表示属性。





```
interface CharStorage{  
    void put(char c);  
    char get(); }  
  
class Stack implements CharStorage{  
    char mem[ ]=new char[10];  
    int point=0;  
    public void put(char c) {  
        mem[point]=c;  
        point++;}  
    public char get() {  
        point--;  
        return mem[point]; }}
```



复习与预习

复习

1. 抽象类、接口、多态

预习

内部类

